CSI4004

TEXT MINING

SMART NEWS CATEGORIZATION SYSTEM USING TEXT MINING AND NLP

A CASE STUDY

Submitted by

Shobana Balusamy

23MID0212

Fall Semester 2026–2027

Outcome: Hackathon

https://drive.google.com/drive/folders/1dmVYXmecYoCOpYknjwjcQX5uyqcC1Kan

Abstract

Online platforms publish and generate enormous volumes of textual information every day, including news articles, blogs, research discussions, and social media posts. Organizing this information into relevant categories manually is a time-consuming, inefficient, and inconsistent process. To address this challenge, automated text classification techniques can be used to identify the subject of a document and categorize it accurately. This case study focuses on the development of a Smart News Categorization System using Natural Language Processing (NLP) and supervised machine learning algorithms.

The proposed system follows an end-to-end text classification approach, beginning with dataset collection and preprocessing, followed by feature extraction, model training, evaluation, and deployment through an interactive web application. The system uses the Term Frequency–Inverse Document Frequency (TF-IDF) technique to convert textual content into numerical feature vectors that machine learning algorithms can process. Three supervised learning algorithms, namely Multinomial Naive Bayes, Logistic Regression, and Linear Support Vector Classifier (Linear SVC), are implemented and compared to identify the most suitable model for news categorization.

The experiment is conducted using the 20 Newsgroups dataset, which contains approximately 18,846 discussion documents distributed across 20 predefined topic categories. The dataset's original training and testing split is maintained to ensure consistency in model evaluation. During preprocessing, unnecessary content such as headers, footers, and quoted text is removed to minimize the influence of irrelevant information and metadata. Additional text-cleaning operations are performed to improve the quality of the input data and support effective feature extraction.

The performance of the trained models is evaluated using multiple classification metrics, including accuracy, macro-averaged precision, recall and F1-score, weighted F1-score, and confusion matrices. Prediction latency is also measured to understand the computational efficiency of each algorithm. These evaluation measures provide a comparative understanding of the models' classification performance, their ability to handle different categories, and their suitability for real-time applications. Actual numerical results are generated through the executable Jupyter Notebook rather than assumed or fabricated.

Furthermore, the project incorporates a Streamlit-based interactive web application that allows users to enter or paste news-related text and obtain its predicted category. This interface demonstrates how the trained machine learning model can be integrated into a practical application, making automated text categorization accessible and user-friendly.

Overall, this case study demonstrates the practical application of NLP and supervised machine learning in automating text classification. It provides insights into the effectiveness of different classification algorithms, the importance of text preprocessing and feature extraction, and the challenges associated with model generalization and classification accuracy. The project also identifies potential future enhancements, such as incorporating larger and more diverse datasets, exploring advanced deep learning models, and extending the system to support real-time news classification.

Keywords: *Text Mining, Natural Language Processing (NLP), Text Classification, TF-IDF, News Categorization, Multinomial Naive Bayes, Logistic Regression, Linear Support Vector Classifier, Machine Learning, Streamlit.*

1. Introduction

1.1 Background

The rapid growth of the internet and digital communication platforms has resulted in the continuous generation of large amounts of textual information. Online newspapers, blogs, social media platforms, and discussion forums publish thousands of articles and posts every day, covering different subjects such as science, technology, education, politics, sports, and business. Managing and organizing such a large volume of information manually has become increasingly difficult and time-consuming.

To overcome this challenge, Natural Language Processing (NLP) techniques are used to automate the process of understanding, analyzing, and categorizing textual information. One of the important applications of NLP is text classification, which involves assigning predefined categories or labels to text documents based on their content. A text classification model learns patterns from previously labeled documents and uses this knowledge to predict the appropriate category for new and unseen documents.

The Smart News Categorization System applies supervised machine learning algorithms to automatically identify the topics of textual documents. By analyzing the words and expressions present in a document, the system can determine its most relevant category, reducing the need for manual classification and improving information organization.

1.2 Importance of text mining

Text mining is an important process in data science that involves extracting meaningful information and patterns from unstructured textual data. Unlike numerical datasets, text data does not naturally contain fixed numerical attributes that machine learning algorithms can directly understand. Therefore, text must first be cleaned, processed, and transformed into a suitable numerical representation.

In this project, the Term Frequency–Inverse Document Frequency (TF-IDF) technique is used for feature extraction. It converts text documents into numerical vectors by assigning importance to words based on their frequency within a document and their occurrence across the entire

collection of documents. Words that are frequently used in a particular document but less common across the dataset receive higher importance.

The system also considers two types of textual features:

- Unigrams: Individual words that help identify important terms and keywords within a document.
- Bigrams: Combinations of two consecutive words that help capture short phrases and contextual relationships between words.

These features help machine learning models recognize meaningful patterns in textual content. Text mining therefore plays a significant role in improving classification performance, reducing manual effort, and enabling automated analysis of large text collections.

1.3 Case study scope

The primary objective of this case study is to design and implement a Smart News Categorization System using NLP and supervised machine learning techniques. The project covers the complete text classification workflow, starting from dataset preparation and ending with the deployment of an interactive prediction application.

The implementation includes the following major stages:

1. Dataset Collection: Loading the 20 Newsgroups dataset containing approximately 18,846 discussion documents distributed across 20 predefined topics.
2. Data Exploration: Examining the dataset structure, identifying available categories, and understanding the distribution of documents.
3. Text Preprocessing: Removing unnecessary headers, footers, quoted text, and other irrelevant content to improve data quality.
4. Feature Extraction: Applying TF-IDF vectorization with unigram and bigram features to convert textual documents into numerical representations.
5. Model Development: Training and comparing Multinomial Naive Bayes, Logistic Regression, and Linear Support Vector Classifier algorithms.

6. Model Evaluation: Measuring classification performance using accuracy, precision, recall, F1-score, confusion matrices, and prediction latency.

7. Model Deployment: Saving the trained model and integrating it into a Streamlit-based web application for interactive text categorization.

The project focuses on understanding the performance of traditional machine learning algorithms and demonstrating their practical application in automated text classification. It is developed as an educational prototype rather than a commercial news recommendation or ranking platform.

1.4 Dataset-label clarification

The dataset selected for this study is the 20 Newsgroups dataset, a widely used benchmark dataset for text classification and machine learning experiments. It consists of discussion documents collected from 20 different newsgroups, covering topics such as computer graphics, hardware, software, science, religion, politics, sports-related discussions, and other subjects.

It is important to clarify that the original dataset does not contain exactly four predefined journalism categories such as Sports, Technology, Business, and Politics. Instead, it provides 20 original topic labels. Therefore, this implementation retains the original 20 categories to preserve the dataset's structure and avoid introducing inaccurate or arbitrary label conversions.

The classification system consequently predicts one of the 20 original discussion topics based on the input text. If a future implementation requires only four broad news categories, a different dataset containing those explicitly defined labels would be more appropriate.

1.5 Structure of the report

This report is organized into different sections to explain the development and evaluation of the Smart News Categorization System systematically.

- Introduction: Presents the background, importance, scope, dataset, and objectives of the project.

- Literature Review: Discusses existing research and approaches related to NLP, text mining, and machine learning-based text classification.

- Problem Definition and Objectives: Explains the classification problem and identifies the major goals of the proposed system.
- Methodology: Describes dataset collection, preprocessing, TF-IDF feature extraction, model training, and implementation procedures.
- Experimental Results and Discussion: Presents the actual model evaluation metrics, performance comparisons, confusion matrices, and error analysis generated by the notebook.
- System Implementation: Explains the integration of the selected model with the Streamlit web application.
- Conclusion and Future Scope: Summarizes the project outcomes, limitations, and possible improvements.

The accompanying Jupyter Notebook serves as the primary source for all experimental results, evaluation metrics, and generated visualizations. This ensures that the findings presented in the report are supported by actual model execution rather than assumed or fabricated results.

2. Literature Review

The literature from 2021–2026 reflects continued development in text classification, from sparse lexical features and conventional classifiers to deep neural networks and pretrained transformer models. The following review is focused on methods relevant to a moderate-scope, reproducible student project.

2.1 Traditional and deep text-classification methods

Li et al. (2022) survey text classification approaches spanning traditional machine learning and deep learning. Their review is useful for positioning TF-IDF with linear classifiers as a strong, interpretable baseline before moving to more computationally demanding models. Gasparetto et al. (2022) also provide a broad survey of text classification algorithms, discussing text representation, labeling, model families, datasets, and evaluation. These reviews support a staged experimental design: first establish a transparent baseline, then compare model choices using a consistent evaluation protocol.

2.2 News categorization

Kishore, Sujatha, and Haripriya (2022) discuss NLP-based news classification and the need to organize large amounts of online news content. Their paper reinforces the practical motivation for automatic categorization. In the current case study, the system is kept deliberately lightweight so that the effects of feature extraction and classifier selection can be measured and explained in a classroom setting.

2.3 Preprocessing and representation choices

Siino, Tinnirello, and La Cascia (2024) examine whether common preprocessing steps remain useful across traditional classifiers and transformer models. Their analysis emphasizes that preprocessing effects depend on the model and dataset. This cautions against treating stop-word removal or normalization as universally beneficial. The present implementation uses a clearly documented preprocessing configuration and keeps it consistent across the three models, enabling a fair comparison of classifier performance under the same features.

2.4 Transformer-based classification

Recent surveys of pretrained language models describe how transformer-based systems can represent context beyond simple word counts. The 2024 IEEE Access survey by Minaee and colleagues (as indexed by IEEE Xplore) discusses text-classification applications, accuracy, cost, and safety considerations. A 2025 survey in Neurocomputing reviews classification based on pretrained language models. These works indicate that transformers are an important extension, but also motivate measuring resource use and avoiding the assumption that a larger model is automatically the best choice for every task.

2.5 Research gap and motivation

A student implementation benefits from a reproducible baseline, a consistent held-out evaluation, visible error analysis, and a usable demonstration interface. Many high-level discussions of classification do not, by themselves, provide an immediately runnable teaching artifact tailored to a specific course. This case study addresses that practical gap by connecting the data pipeline, comparative evaluation, and an interactive prototype. It does not claim a new classification algorithm; its contribution is an integrated and auditable implementation suitable for learning and demonstration.

2.6 Summary of reviewed literature

Source	Focus	Relevance to this study
Li et al. (2022)	Survey from traditional to deep-learning text classification	Supports comparing a conventional baseline with alternative classifiers.
Gasparetto et al. (2022)	Survey of text-classification algorithms and evaluation	Informs feature representation, evaluation metrics, and model comparison.
Kishore et al. (2022)	NLP for news classification	Provides context for the application domain.
Siino et al. (2024)	Impact of preprocessing across model types	Motivates documenting and controlling preprocessing choices.
Minaee et al. / IEEE Access (2024)	Survey of transformer-based text classification	Frames accuracy, computational cost, and responsible deployment trade-offs.
Survey in Neurocomputing (2025)	Pretrained language models for text classification	Identifies transformer models as a possible future extension.

3. Problem Statement

Large collections of textual documents are difficult to organize manually and may be inconsistently labeled when volumes grow. The problem addressed is to learn a text classifier that assigns a previously unseen document to one of the corpus's predefined topic categories while achieving useful predictive performance and reasonable inference speed.

The task is formalized as supervised multiclass classification. Given a labeled training set $D = \{(x_i, y_i)\}$, where x_i is a text document and y_i is one of the 20 original topic labels, the system learns a function $f(x)$ that predicts a topic for new text. The model should be evaluated on held-out data rather than only on its training examples.

3.1 Real-world significance

Automatic categorization can support document browsing, archive organization, content discovery, and triage. However, an incorrect category can mislead users. Therefore, performance must be measured per class as well as overall, and the model should be treated as an aid rather than an infallible authority.

4. Objectives

1. To load and document a public text corpus with known category labels.
2. To inspect class balance and document-length patterns before training.

3. To transform text into TF-IDF features using unigrams and bigrams.
4. To train and compare Multinomial Naive Bayes, Logistic Regression, and Linear SVM.
5. To evaluate models using accuracy, macro precision/recall/F1, weighted F1, confusion matrices, and prediction latency.
6. To examine misclassified examples and discuss likely sources of error.
7. To save the best-performing trained pipeline and integrate it with a Streamlit prediction interface.
8. To identify dataset limitations and propose feasible future improvements.

5. Methodology / Proposed Techniques

5.1 Dataset description

The 20 Newsgroups dataset distributed through scikit-learn contains approximately 18,846 text documents across 20 newsgroup topics. The documented split contains 11,314 training documents and 7,532 test documents when using the full standard dataset. The split is based on posting date, which gives a temporally separated evaluation rather than a random split of all documents. The notebook uses the predefined split and removes headers, footers, and quoted text using the dataset loader's options.

Attribute	Description
Source	scikit-learn fetch_20newsgroups dataset loader
Input	Text document / discussion post
Target	One of 20 original newsgroup categories
Training set	11,314 documents in the full standard split
Test set	7,532 documents in the full standard split
Data type	Unstructured English text
Important caveat	Discussion posts, not current professional news articles

5.2 Software and tools

Python is used for data handling and modelling. scikit-learn supplies the dataset loader, TF-IDF vectorizer, classifiers, and metrics. Pandas structures metric tables, Matplotlib and Seaborn support visualizations, joblib serializes the trained pipeline, and Streamlit provides the

interactive user interface. The notebook records the scikit-learn version so that the environment can be documented.

5.3 Data preprocessing

9. Load the official training and test subsets with a fixed random state.
10. Remove headers, footers, and quoted text to reduce metadata leakage.
11. Use TF-IDF vectorization to transform tokens into numerical features.
12. Convert text to lowercase and remove English stop words.
13. Use unigrams and bigrams to represent single terms and short phrases.
14. Ignore terms occurring in fewer than two documents and cap vocabulary size for practical resource use.
15. Fit the vectorizer only on training text through a scikit-learn Pipeline, then transform test text with the fitted vectorizer.

The chosen preprocessing is a controlled baseline rather than a claim that every step is optimal. A future experiment could compare preprocessing variants, stemming or lemmatization, character n-grams, and different minimum document frequencies.

5.4 TF-IDF representation

Term Frequency–Inverse Document Frequency gives higher weight to words that are useful for distinguishing a document from the broader collection. In simplified form, TF-IDF combines a term-frequency value with an inverse-document-frequency value. The scikit-learn vectorizer applies its documented weighting and normalization. Bigrams may capture phrases such as "graphics card" or "space shuttle," while unigrams retain individual topic terms.

5.5 Classification algorithms

Model	Role	Strengths and limitations
Multinomial Naive Bayes	Probabilistic baseline for word-feature classification	Fast and simple; assumes conditional independence of features.
Logistic Regression	Linear discriminative classifier	Often effective for sparse text; regularization and iteration limits affect results.
Linear Support Vector Classifier	Margin-based linear classifier	Strong conventional text baseline; does not directly provide calibrated probabilities.

5.6 Experimental design

All models use the same train/test data and the same TF-IDF configuration. The test set is held out until final evaluation. Model selection in the notebook is based on macro-F1, which gives each class equal weight and is informative when class frequencies differ. Weighted-F1 is also reported to reflect the class distribution. Training and prediction times are measured on the executing machine; these times should not be treated as universal benchmark values.

5.7 Evaluation metrics

- Accuracy: proportion of all test documents assigned the correct class.
- Precision: among documents predicted as a class, the proportion that truly belong to that class.
- Recall: among documents belonging to a class, the proportion correctly retrieved.
- F1-score: harmonic mean of precision and recall.
- Macro-F1: arithmetic mean of class-level F1-scores, giving equal importance to every class.
- Weighted-F1: class-level F1 averaged using each class's support.
- Confusion matrix: shows which true classes are confused with which predicted classes.
- Prediction latency: elapsed prediction time divided by the number of test documents in a measured batch.

5.8 Workflow

Step	Activity	Output
1	Load dataset and labels	Training/test documents and topic names
2	Inspect data	Class counts and document-length chart
3	Vectorize text	Sparse TF-IDF feature matrix
4	Train classifiers	Three fitted model pipelines
5	Evaluate	Metric table and classification reports
6	Analyze errors	Confusion matrix and misclassified examples
7	Select model	Highest observed Macro-F1 model
8	Save and deploy	joblib pipeline and Streamlit app

5.9 Pseudocode

INPUT: labeled training documents, labeled test documents

LOAD the predefined corpus split

REMOVE headers, footers, and quoted text

FOR each classifier in {Naive Bayes, Logistic Regression, Linear SVM}:

  CREATE a pipeline containing TF-IDF and the classifier

  FIT the pipeline using training documents only

  PREDICT labels for test documents

  COMPUTE accuracy, macro-F1, weighted-F1, and per-class metrics

  MEASURE training and prediction time

SELECT the classifier with the highest measured macro-F1

SAVE the selected pipeline, label names, and model metadata

DISPLAY a user-entered document's predicted original topic

OUTPUT metric tables, plots, and prediction interface

5.10 Application design

The Streamlit application loads the saved pipeline, accepts text through a text area, checks that the input is not trivially short, and displays the predicted topic. For models that support predict_proba, it can display the top class scores; for Linear SVM, the app clearly states that a decision score is not a calibrated probability. The application includes a notice that the model uses original newsgroup labels and is an educational prototype.

6. Results and Discussion

This section must be completed after running the accompanying notebook. No model scores have been fabricated in this report. The notebook generates the actual comparison table, class-level report, confusion matrix, error examples, and latency table using the installed environment and the downloaded dataset.

Best Model Selection

Logistic Regression was selected as the final model because it achieved the highest Macro-F1 and Weighted-F1 scores among the three algorithms. Although Linear SVM achieved the same

accuracy, Logistic Regression demonstrated slightly better performance across the F1-score metrics.

6.1 Model comparison

Model	Accuracy	Macro-F1	Weighted-F1	Training time (s)	Mean prediction time (ms/document)
Multinomial Naive Bayes	0.6776	0.6462	0.6641	10.0735	0.2792
Logistic Regression	0.7006	0.6888	0.6991	50.7093	0.3106
Linear SVM	0.7006	0.6883	0.6985	10.7645	0.3410

6.2 Interpretation of findings

After executing the notebook, identify the model with the highest macro-F1 and explain whether its accuracy and weighted-F1 tell the same story. If the model with the best macro-F1 is not the fastest, discuss the trade-off between class-balanced performance and latency. Avoid stating that one model is superior without reference to the measured values.

6.3 Class-level results

Use the classification report to identify topics with low recall or precision. Low recall means that examples from a topic are often assigned to another topic; low precision means that many documents assigned to a topic actually belong elsewhere. Related topics may share vocabulary, while short or ambiguous posts may not contain enough context for reliable categorization.

6.4 Confusion matrix and error analysis

The confusion matrix should be used to locate systematic confusions. Review representative misclassified examples exported by the notebook and consider whether the text is genuinely ambiguous, whether the message contains insufficient topical vocabulary, or whether the original newsgroup label is not well represented by its body text. The dataset can contain noisy and historical discussion content, so not every error necessarily indicates a model implementation defect.

6.5 Practical implications

A successful prototype demonstrates that conventional NLP pipelines can turn raw text into searchable topic predictions with modest software requirements. The interface can support demonstrations and manual exploration. Before practical deployment, the model would need validation on current news articles, monitoring for topic drift, and a clear mechanism for handling uncertain or out-of-domain text.

6.6 Limitations

- The corpus contains historical discussion posts rather than current professional news articles.
- The original topic labels do not match exactly the four broad categories Sports, Technology, Business, and Politics.
- Bag-of-words TF-IDF features do not fully represent word order, context, irony, or nuanced semantics.
- Removing headers, footers, and quotes uses heuristics and may also remove useful text.
- Latency depends on hardware, software versions, and workload.
- The application predicts a topic even for some text that may be outside the training distribution; an abstention or uncertainty mechanism is not implemented.

7. Conclusion

This case study presents a reproducible end-to-end framework for multiclass text categorization using TF-IDF features and three conventional machine-learning classifiers. It connects dataset inspection, preprocessing, training, evaluation, error analysis, model serialization, and an interactive Streamlit interface. The approach is appropriate for a moderate-scope CSI4004 project because each stage can be inspected and explained, and the results can be regenerated from code. The final empirical conclusion must be written after execution: state the best model by the selected metric, provide its actual test scores, describe the principal confusion patterns, and report measured latency. The project should not claim that the model categorizes current news into four journalism categories unless a correspondingly labeled dataset is used.

8. Future Directions

16. Use AG News or another labeled news corpus if the required categories are World, Sports, Business, and Sci/Tech.
17. Compare word-level TF-IDF with character n-grams and alternative preprocessing configurations.
18. Add cross-validation on training data for more robust model selection while preserving a final untouched test set.
19. Evaluate pretrained transformer models such as DistilBERT or BERT and compare accuracy, latency, memory, and deployment cost.
20. Add a confidence threshold or abstention option for ambiguous and out-of-domain text.
21. Expand error analysis with examples from each class and evaluate whether class imbalance affects macro-F1.
22. Package the model and app in a reproducible environment, record dependency versions, and add automated tests.
23. Assess data rights, privacy, bias, and domain shift before using the prototype on live content.

9. References

Gasparetto, A., Marcuzzo, M., Zangari, A., & Albarelli, A. (2022). A survey on text classification algorithms: From text to predictions. Information, 13(2), 83. https://doi.org/10.3390/info13020083

Kishore, K., Sujatha, V., & Haripriya, A. (2022). A comprehensive analysis of news classification using NLP. Mathematical Statistician and Engineering Applications, 71(1), 248–252. https://doi.org/10.17762/msea.v71i1.1475

Li, Q., Peng, H., Li, J., Xia, C., Yang, R., Sun, L., & He, L. (2022). A survey on text classification: From traditional to deep learning. ACM Transactions on Intelligent Systems and Technology, 13(2). https://doi.org/10.1145/3495162

scikit-learn developers. (n.d.). The 20 newsgroups text dataset. scikit-learn documentation. https://scikit-learn.org/stable/datasets/real_world.html

scikit-learn developers. (n.d.). fetch_20newsgroups. scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_20newsgroups.html

Siino, M., Tinnirello, I., & La Cascia, M. (2024). Is text preprocessing still worth the time? A comparative survey on the influence of popular preprocessing methods on Transformers and traditional classifiers. Information Systems, 121, 102342. https://doi.org/10.1016/j.is.2023.102342

A Survey of Text Classification With Transformers: How Wide? How Large? How Long? How Accurate? How Expensive? How Safe? (2024). IEEE Access, 12, 6518–6531. https://doi.org/10.1109/ACCESS.2024.3349952

A survey of text classification based on pre-trained language model. (2025). Neurocomputing, 616, 128921. https://doi.org/10.1016/j.neucom.2024.128921

Appendix A. Running the Prototype

24. Install the packages listed in requirements.txt.
25. Open CSI4004_Smart_News_Categorization.ipynb in Jupyter or Google Colab.
26. Run all cells in order; allow the first dataset download to complete.
27. Record the metrics and use the notebook's generated plots in Section 6.
28. Ensure outputs/news_classifier.joblib exists.
29. Run `streamlit run app.py` from the project directory.
30. Paste an article or post and click Classify article to view the predicted topic.

Github Link: https://github.com/Shobana0212/Text-Mining-Case-Study.git